



CONSTRUINDO A TELA DE APRESENTAÇÃO

HTML Semântico em Vue.js
Aula 1



Professora Ma. Tatiana Aparecida de Almeida

2. SUMÁRIO DA AULA



1 O que vamos aprender hoje

2 UX e Padrão de Cores

3 Conteúdo: Textos e Imagens

4 Prototipando no Figma

5 Revisão: HTML Semântico

6 Estilizando com Tailwind CSS



7. Mão na Massa (Código)

3. O QUE VAMOS APRENDER HOJE



O objetivo desta aula é entender **todo o processo** de criação de uma tela: desde o planejamento visual até a codificação final.



1. Planejamento Visual: Como escolher cores (UX), definir os textos e criar um protótipo rápido no Figma.



2. Estrutura Correta: Revisar como o HTML Semântico ajuda a organizar a página para o navegador e para o Google.



3. Estilização Moderna: Usar o Tailwind CSS para deixar a página bonita escrevendo menos código.



Resultado Esperado: A Landing Page do projeto CTI Insights codificada no nosso projeto Vue.js!

4. UX E PADRÃO DE CORES



UX: Guiando a Atenção

Em **UX (Experiência do Usuário)**, as cores indicam prioridades: onde clicar e o que é secundário.

Identidade Visual

O projeto deve refletir a marca. Para o CTI Insights (SENAI) usamos as cores institucionais: Vermelho e Cinza/Preto.

A Regra 60-30-10

Dica de design: **60%** dominante (fundos claros), **30%** secundária (textos/menus), **10%** destaque (botões/alertas).

Paleta do Projeto

Fundo (60%)

#F8F9FA (Tailwind: bg-gray-50)

Textos/Menus (30%)

#333333 (Tailwind: text-gray-800)

Destaque (10%)

#E3000F (Tailwind: bg-red-600)

5. CONTEÚDO: TEXTOS E IMAGENS



Regra de Ouro: Content-First (Conteúdo Primeiro)

Nunca comece desenhando telas vazias. O design deve acomodar o conteúdo, e não o contrário.

1

A Dor do Usuário

O que o sistema resolve? No nosso caso: a dificuldade de analisar planilhas complexas. Todo o texto deve focar em resolver essa dor.

2

Título Principal (Hero)

Deve ser a primeira coisa que o usuário lê. Precisa ser claro e direto.
Ex: "Transforme Dados em Decisões".

3

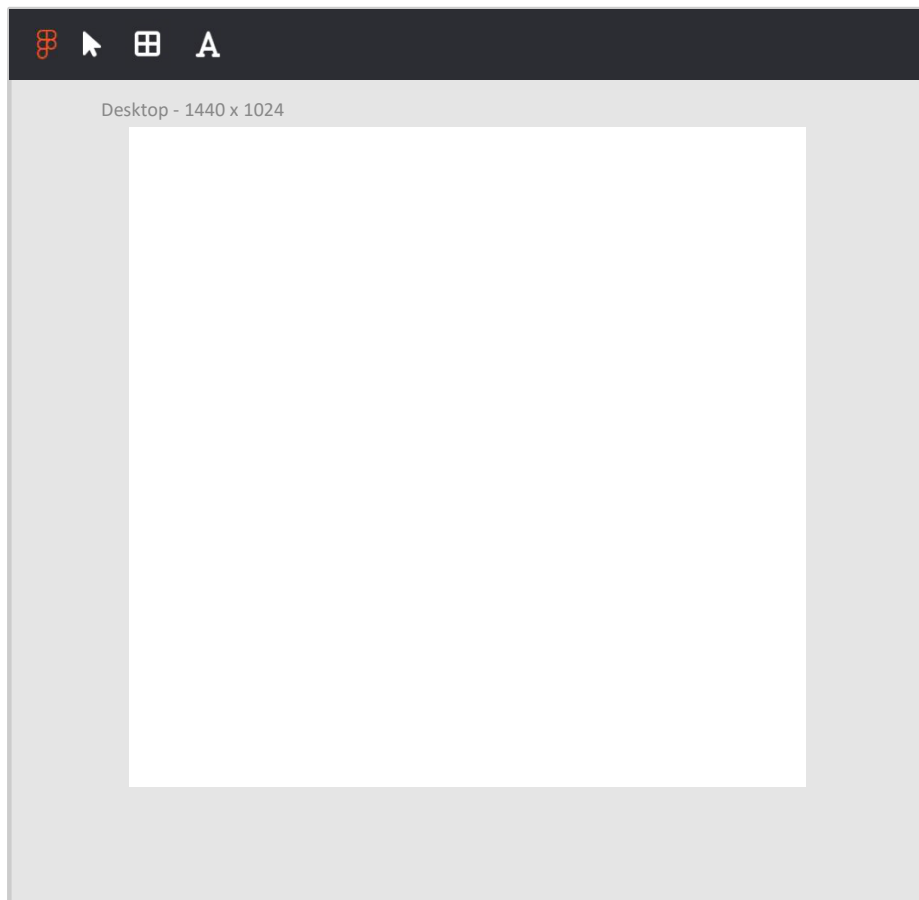
Funcionalidades

Crie textos curtos para os "cards" de explicação. Ninguém lê textos longos na internet.
Ex: "Upload Simples", "Gráficos Automáticos".



Dica de Produtividade: Você pode usar ferramentas de IA (como ChatGPT ou Claude) para ajudar a criar o *Copywriting* (textos persuasivos) da sua página antes de ir para o Figma.

6. PROTOTIPANDO NO FIGMA



1

Crie o Frame (Tela)

Aperte a tecla **F** no Figma e escolha o tamanho **Desktop (1440px)**. Esse é o tamanho padrão para telas de computador modernas.

2

Configure o Grid (Grade)

Adicione um *Layout Grid* de **12 colunas**. Isso é essencial para manter tudo alinhado e facilita muito na hora de passar para o código.

3

Faça o Wireframe (Esqueleto)

Antes de colocar cores e imagens, use retângulos cinzas para marcar onde ficará o Header, o Hero e os Cards. Isso ajuda a focar na estrutura primeiro.

7. DICAS DE DESIGN NO FIGMA



Auto Layout

Shift + A

A melhor ferramenta do Figma

Funciona exatamente como o **Flexbox do CSS**. Ele organiza os elementos automaticamente (lado a lado ou em coluna) e mantém o espaçamento perfeito. Use em tudo: botões, menus e cards.



Componentes

Ctrl+Alt+K

Crie uma vez, use várias

Transforme botões e cards em componentes. Se você precisar mudar a cor do botão principal depois, você altera apenas no componente "Mestre" e todos os outros botões do projeto mudam sozinhos.



Espaços

Sistema de 8

Use a regra dos Múltiplos de 8

Para margens e paddings, use sempre múltiplos de 4 ou 8 (ex: 8px, 16px, 24px, 32px, 64px). **Nunca use números quebrados** como 13px ou 27px. O Tailwind CSS usa essa mesma lógica por padrão!

8. LINKS ÚTEIS: CINTO DE UTILIDADES



Paletas de Cores

Coolors: coolors.co (Gerador rápido)

Color Hunt: colorhunt.co (Paletas prontas)

Imagens Gratuitas

Unsplash: unsplash.com (Fotos de alta qualidade)

Freepik: freepik.com (Fotos e ilustrações)

Ícones

FontAwesome: fontawesome.com (Usado no nosso projeto)

Phosphor: phosphoricons.com (Ícones modernos)

Inspiração de Design

Dribbble: dribbble.com (Ideias de UI/UX)

Behance: behance.net (Projetos completos)

1. O QUE É HTML SEMÂNTICO?



Semântica significa "significado". HTML Semântico é usar as tags corretas para descrever o **propósito** do conteúdo, e não apenas a sua aparência.

Por que usar?

-  **SEO (Google):** Os motores de busca entendem melhor a estrutura da sua página, melhorando o ranqueamento.
-  **Acessibilidade:** Leitores de tela para deficientes visuais conseguem navegar pela página com muito mais facilidade.
-  **Manutenção:** O código fica mais limpo e fácil de ler para outros programadores (e para você no futuro!).

❌ Não Semântico ("Div Soup")

```
<div id="cabecalho">
<div class="menu">...</div>
</div>
<div id="conteudo-principal">
<div class="noticia">...</div>
</div>
<div id="rodape">...</div>
```

✅ Semântico (Correto)

```
<header>
<nav>...</nav>
</header>
<main>
<article>...</article>
</main>
<footer>...</footer>
```

2. TAGS SEMÂNTICAS PRINCIPAIS



<header>

O **cabeçalho** da página. Geralmente contém a logo do sistema, o título principal e, muitas vezes, o menu de navegação.



<nav>

Usado exclusivamente para agrupar os principais **links de navegação** (menus). Fica frequentemente dentro do header.



<main>

O **conteúdo principal** e único da página. Só deve existir um <main> por página, excluindo cabeçalhos, rodapés e sidebars.



<section>

Uma **seção genérica** que agrupa conteúdos relacionados. Exemplo: Seção "Sobre nós", Seção "Funcionalidades", Seção "Contato".



<article>

Conteúdo **independente** que faria sentido se lido fora da página. Ex: Um post de blog, um comentário de fórum, uma notícia.



<footer>

O **rodapé** da página. Contém informações de direitos autorais, links secundários, contatos e redes sociais da empresa.

TAILWIND CSS: ESTILIZANDO RÁPIDO



⇒ Exemplo: Header com Tailwind

```
<header class="bg-white shadow-md flex justify-between p-4">

<div class="flex items-center gap-2">



<h1 class="text-xl font-bold text-gray-800">CTI</h1>

</div>

<nav class="flex gap-4">

<a href="#" class="text-red-600 hover:underline">Login</a>

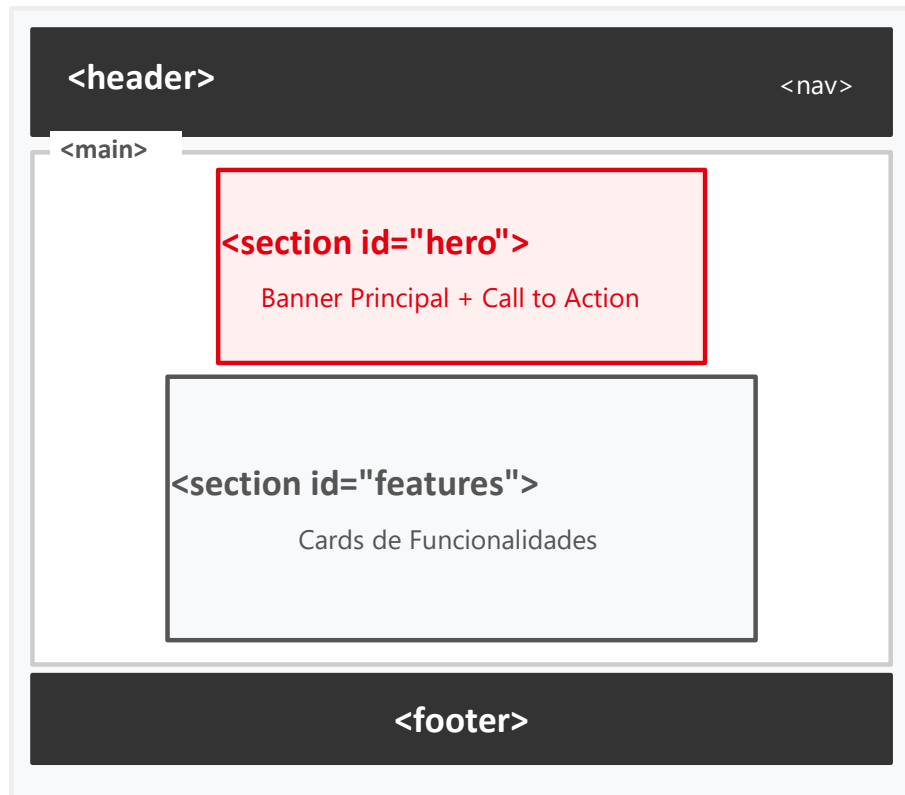
</nav>

</header>
```

O que é o Tailwind?

É um framework CSS baseado em **classes utilitárias**. Em vez de criar um arquivo `style.css` separado, você aplica o estilo diretamente na tag HTML.

3. ESTRUTURA DA LANDING PAGE



O Mapa do Projeto: Antes de programar, desenhamos o "esqueleto" da página usando as tags semânticas para organizar o layout.

<header> Ficará no topo. Conterá a logo do **CTI Insights** e a tag <nav> com o botão para a tela de Login.

<main> Engloba todo o conteúdo central da página de apresentação.

<section> Dividimos o conteúdo em duas seções principais: **1. Hero** — a primeira área visível com título de impacto; **2. Features** — a área que explica os benefícios do sistema.

<footer> O rodapé com os direitos autorais (Copyright) e links institucionais.

4. PARTE 1: HEADER E NAVEGAÇÃO



src/pages/LandingPage.vue

```
<template>

<header class="cabecalho-principal">

  <div class="logo-container">

    <h1>CTI Insights</h1>

  </div>

  <nav class="menu-navegacao">
    <router-link to="/">Início</router-link>
    <router-link to="/sobre">Sobre</router-link>
    <router-link to="/login" class="btn-login">Login</router-link>
  </nav>

</header>
</template>
```

H A tag <header>

Envolve toda a parte superior da página. Ela agrupa a identidade visual (logo e título) e o menu de navegação em um único bloco semântico.

🗏 A tag <nav>

Indica aos navegadores e leitores de tela que os links dentro dela são os **links principais de navegação** do site.

🔗 <router-link>

No Vue.js, usamos <router-link> em vez da tag <a> padrão. Isso permite navegar entre as telas (como ir para o Login) **sem recarregar a página inteira**.

5. PARTE 2: SEÇÃO HERO



✓ LandingPage.vue (Continuação)

```
<main>
<section class="hero">

  <div class="hero-content">

    <h1>Transforme Dados em Decisões</h1>

    <p>CTI Insights gera dashboards interativos automaticamente.</p>

    <router-link to="/login" class="btn-primary">

      Acessar o Sistema

    </router-link>

  </div>

  <div class="hero-image">

  </div>
</section>
```

★ A tag <main>

Conteúdo principal da página vai dentro de <main> — único por página.

≡ A tag <section>

Agrupa a área "Hero" como uma seção temática clara.

H Hierarquia de Títulos

Use um único <h1> por página para indicar o assunto principal.

PARTE 3: SEÇÃO DE FUNCIONALIDADES



✓ LandingPage.vue (Template)

```
<section id="funcionalidades">

<h2>Por que usar o CTI Insights?</h2>

<div class="cards-container">

<article class="card">

<h3>Upload Simples</h3>

<p>Envie suas planilhas com um clique.</p>

</article>

<article class="card">

<h3>Gráficos Automáticos</h3>

<p>Visualize dados em tempo real.</p>

</article>
```

A tag <section>

Usamos <section> para agrupar um bloco de conteúdo que tem um tema em comum. Aqui, o tema é "Funcionalidades". Toda section deve ter um título (como o <h2>).

A tag <article>

Cada "card" de funcionalidade é um <article>. Usamos essa tag porque cada funcionalidade é uma informação independente que faz sentido por si só.

A tag <div>

Ainda usamos <div>! Mas apenas para fins de layout e estilização (como o container dos cards), e não para dar significado ao conteúdo.

7. PARTE 4: FOOTER



✓ LandingPage.vue (Rodapé)

```
<template>
<footer class="rodape">

<div class="rodape-conteudo">

<p>&copy; 2026 CTI Insights. Todos os direitos reservados.</p>

<nav class="rodape-links">

<a href="#">Termos de Uso</a>

<a href="#">Privacidade</a>

</nav>

</div>
</footer>
</template>
```

A tag <footer>

Representa o rodapé da página. Ideal para direitos autorais, links legais e contatos.

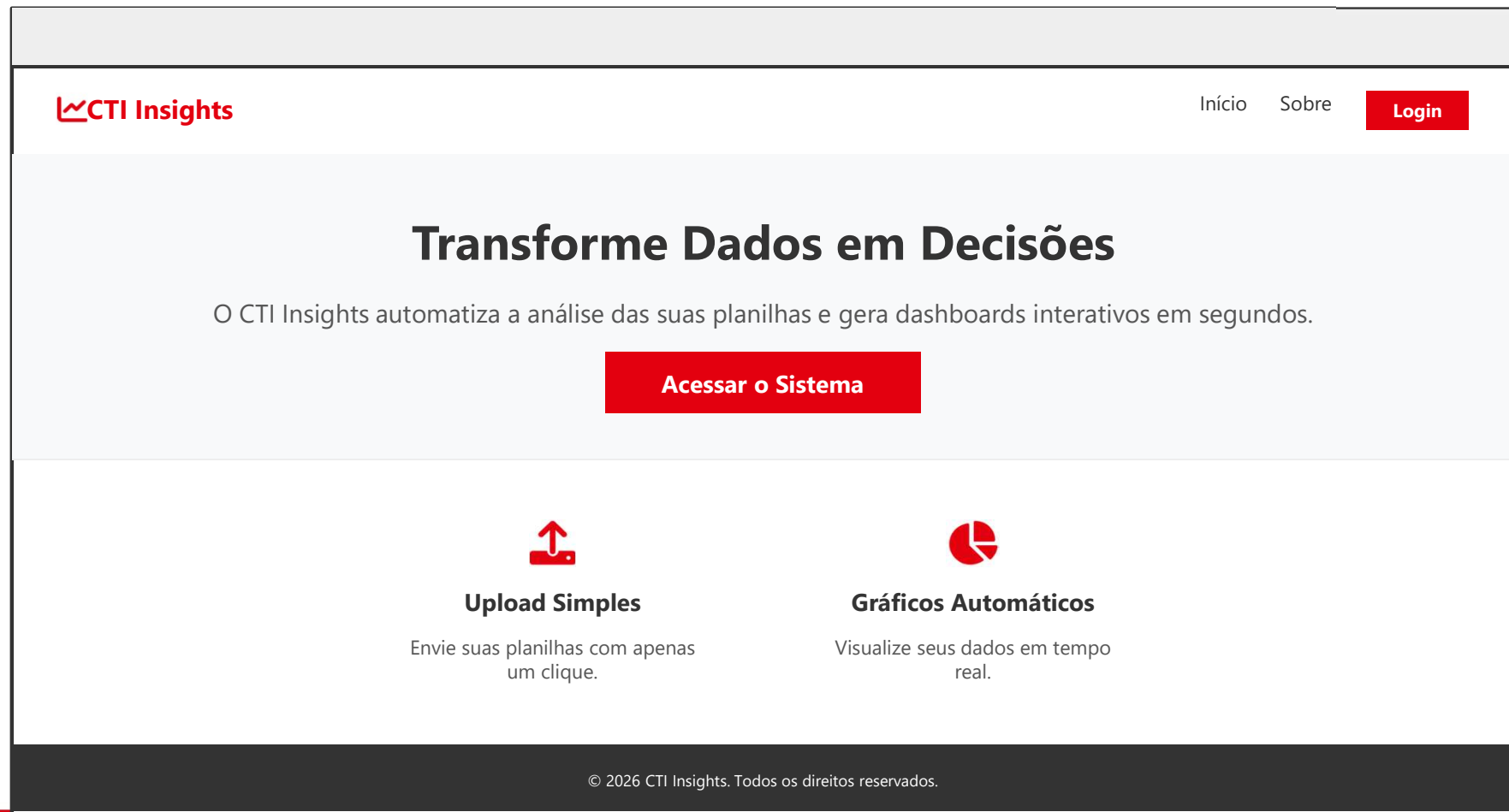
<nav> no Rodapé?

Sim. A tag <nav> pode ser usada no rodapé para agrupar links secundários.

8. RESULTADO FINAL



Juntando o **HTML Semântico** com o **Tailwind CSS**, este é o visual da nossa página:



9. RESUMO



O que aprendemos hoje



HTML Semântico: Fundamental para SEO e acessibilidade. Dá significado à estrutura da página.



Tags: Aplicamos na prática as tags `<header>`, `<main>`, `<section>`, `<article>` e `<footer>`.



Tailwind CSS: Vimos como estilizar rapidamente usando classes utilitárias diretamente no HTML.